



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251193
Nama Lengkap	Cheila Zefanya Wibowo
Minggu ke / Materi	15/Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengertian Rekursif

Fungsi rekursif adalah fungsi yang berisi dirinya sendiri atau fungsi yang mendefinisikan dirinya selama proses eksekusi. Fungsi ini merupakan fungsi matematis yang berulang dan memiliki pola yang terstruktur, namun fungsi ini perlu diperhatikan agar ini dapat berhenti dan tidak menghabiskan memori. Menggunakan fungsi harus hati-hati karena bersifat unlimited loop sehingga menyebabkan program hang up.

Fungsi rekursif memiliki 2 blok penting, yaitu blok yang menjadi titik berhenti dan blok yang memanggil dirinya sendiri. Di dalam rekursif terdapat 2 bagian :

1. **Base Case** : bagian dimana penentu bahwa fungsi rekursif itu berhenti.
2. **Rekursif Case** : bagian dimana terdapat statement yang terus diulang terus menerus hingga mencapai Base Case.

Komponen Penting

Base Case :

```
if n == 0:  
    return 1
```

Recursive Case :

```
else:  
    return n * factorial (n-1)
```

Kelebihan dan Kekurangan

Beberapa kelebihan fungsi rekursif sebagai berikut:

1. Kode lebih elegan, artinya solusi untuk masalah tertentu lebih alami dan mudah dipahami.
2. Breakdown masalah, artinya memecah masalah kompleks menjadi sub masalah yang lebih sederhana.
3. Sesuai untuk struktur data rekursif seperti pohon dan graf.

Beberapa kekurangan fungsi rekursif sebagai berikut :

1. Overhead memori, artinya setiap pemanggilan membutuhkan ruang stack.
2. Potensi stack overflow, artinya untuk rekursi yang sangat dalam.
3. Efisiensi, artinya biasanya program lebih lambat daripada solusi iteratif.
4. Kesulitan debugging, artinya lebih sulit dilacak dibanding solusi iteratif.

Rekursif baik digunakan ketika :

1. Masalah bisa didefinisikan ulang.
2. Struktur data yang digunakan bersifat rekursif, seperti pohon(tree) atau graf.
3. Solusi rekursif lebih mudah dibaca dibanding solusi iteratif.

Bentuk umum fungsi rekursif:

```
def fungsi_rekursif(parameter):  
    if base_case(parameter): # Kondisi berhenti  
        return nilai_dasar  
    else:  
        return operasi  
        (fungsi_rekursif(parameter_termodifikasi), parameter)
```

Contoh 1:

```
1 def hitung_mundur(n):
2     if n == 0:
3         print("Gol")
4     else:
5         print(n)
6         hitung_mundur(n-1)
7
```

Contoh 2:

```
1 def coba2(n):
2     if n == 1 :
3         return 1
4     else:
5         return n+coba2(n-1)
6 print(coba2(10))
7
```

Output :

```
PS C:\pra alpro\belajar> python -
55
```

Contoh 3:

```
1 def coba2(n):
2     if n == 1 :
3         return 1
4     else:
5         return n*coba2(n-1)
6 print(coba2(10))
7
```

Output :

```
PS C:\pra alpro\belaja
3628800
```

Rekursif vs Iteratif

Setiap fungsi rekursif pasti memiliki solusi iteratifnya. Keduanya menghasilkan output yang sama, namun dengan cara yang berbeda. Berikut perbandingannya:

ITERATIF	REKURSIF
Menggunakan loop(for/while)	Memanggil dirinya sendiri
Lebih hemat memori	Lebih boros memori
Mudah didebug	Lebih sulit didebug
Kode lebih panjang	Kode lebih elegan dan singkat

Bentuk Umum dan Studi Kasus

Berikut adalah kerangka dasar penulisan fungsi rekursif dalam python :

```
def nama_fungsi (parameter):  
    if kondisi_berhenti :  
        return nilai_akhir  
    else:  
        return nama_fungsi
```

Perkalian Rekursif

Contoh $2*4 = 2+2+2+2 = 8$. Rumus nya adalah $a*b = a+(a*(b-1))$

```

1  def perkalian (bil1, bil2):
2      if bil2 == bil1 :
3          return bil1
4      else:
5          return bil1 + perkalian(bil1,bil2-1)
6  print(perkalian(2, 4))

```

Bilangan Fibonacci

Di mana setiap bilangan adalah hasil penjumlahan dua bilangan sebelumnya.

Rumus rekursif Fibonacci:

- $F(1) = 1$ dan $F(2) = 1$
- $F(n) = F(n-1) + F(n-2)$

```

1  def fibo(n):
2      if n == 1 or n == 2:
3          return 1
4      else:
5          return fibo (n-1) + fibo(n-2)
6  print(fibo(7))

```

Sumber : Materi 15 Rekursif (eclass)

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link GitHub : https://github.com/chei-cell/71251193_cheila-.git

SOAL 1

Source Code :

```
def cek_prima(n, pembagi=2):  
    if n < 2:  
        return False  
    if pembagi > n ** 0.5:  
        return True  
    if n % pembagi == 0:  
        return False  
    return cek_prima(n, pembagi + 1)  
  
print(cek_prima(5))  
  
print(cek_prima(19))  
  
print(cek_prima(28))
```

Langkah-langkah :

1. Pertama, membuat fungsi dengan nama cek_prima dengan parameter n dan pembagi.
2. Memeriksa apakah nilai n kurang dari 2. Jika kurang dari 2, fungsi mengembalikan nilai false karena tidak bilangan prima.
3. Memeriksa nilai pembagi lebih besar dari kuadrat n. jika iya, fungsi mengembalikan nilai true karna tidak menemukan pembagi.

4. Memeriksa apakah n habis dibagi oleh pembagi menggunakan modulus(%).
5. Jika hasil modulus adalah 0, fungsi akan mengembalikan nilai false karena memiliki factor selain 1 dan dirinya sendiri.
6. Jika hasil modulus tidak habis dibagi, fungsi memanggil dirinya sendiri dengan menambahkan nilai pembagi sebesar 1.
7. Proses pengecekan dilakukan berulang hingga pembagi melebihi akan kuadrat n.
8. Memanggil fungsi cek_prima(5), cek_prima(19), dan cek_prima(28).
9. Menampilkan hasil

Output :

```
PS C:\pra alpro\minggu 15> python -t
True
True
False
```

SOAL 2

Source Code:

```
def palindrome(kata):
    kata = kata.lower().replace(" ", "")

    if len(kata) <= 1:
        return True

    if kata[0] != kata[-1]:
        return False

    return palindrome(kata[1:-1])
```

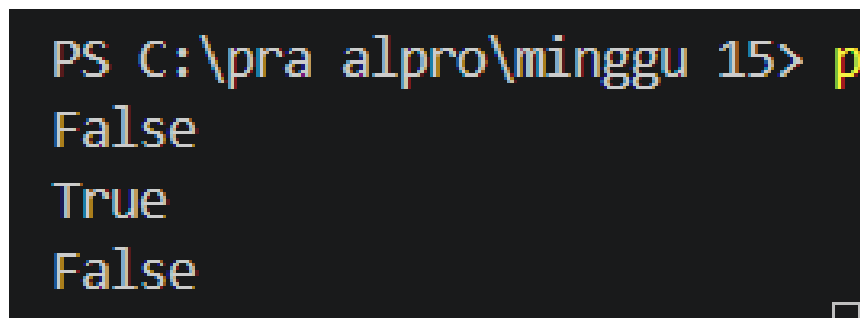


```
print (palindrome ("monyet"))  
print (palindrome ("radar"))  
print (palindrome ("java"))
```

Langkah-langkah :

1. Membuat fungsi bernama palindrome dengan parameter kata.
2. Mengubah semua huruf menjadi kecil menggunakan lower() dan menghapus spasi menggunakan replace(",")
3. Memeriksa apakah panjang kata kurang dari atau sama dengan 1, len(kata) <= 1
4. Jika panjang kata kurang dari atau sama dengan 1, maka akan mengembalikan nilai true
5. Membandingkan karakter pertama dengan karakter terakhir.
6. Jika karakter pertama dan terakhir berbeda, fungsi akan mengembalikan nilai false.
7. Jika karakter pertama sama dengan terakhir, fungsi akan memanggil dirinya sendiri menggunakan kata[1:-1]
8. Melakukan pengecekan dilakukan berulang hingga semua karakter bisa dibandingkan.
9. Memanggil fungsi lagi dan menampilkan hasilnya

Output :



```
PS C:\pra alpro\minggu 15> p  
False  
True  
False
```

SOAL 3

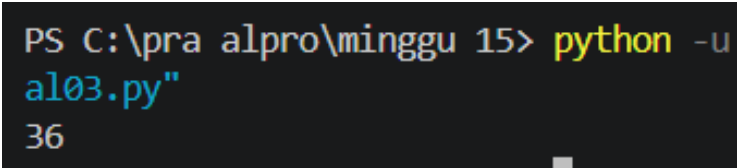
Source Code:

```
def jumlah_ganjil(n):  
    if n == 1:  
        return 1  
  
    return n + jumlah_ganjil(n - 2)  
  
print(jumlah_ganjil(11))
```

Langkah – langkah :

1. Membuat fungsi bernama jumlah_ganjil dengan menggunakan parameter n
2. Periksa apakah nilai n sama dengan 1. Jika sama, fungsi mengembalikan nilai 1.
3. Jika n tidak sama dengan 1, fungsi menjumlahkan nilai n dengan hasil rekursif (jumlah_ganjil(n-2)).
4. Fungsi terus memanggil dirinya sendiri dengan mengurangi nilai n sebesar 2 hingga mencapai 1.
5. Memanggil fungsi jumlah_ganjil(11).
6. Menampilkan hasil

Output :



```
PS C:\pra alpro\minggu 15> python -u  
al03.py"  
36
```

SOAL 4

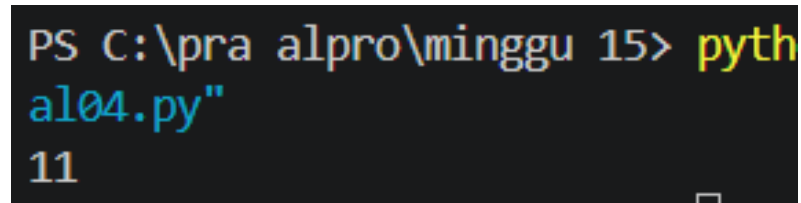
Source Code:

```
def jumlah_digit(n):  
    if n < 10:  
        return n  
  
    return n % 10 + jumlah_digit(n // 10)  
  
print(jumlah_digit(254))
```

Langkah – langkah :

1. Membuat fungsi bernama jumlah_digit dengan menggunakan parameter n
2. Periksa apakah nilai n kurang dari 10 dengan menggunakan if n<10
3. Jika n kurang dari 10, fungsi mengembalikan nilai n hanya karena memiliki satu digit.
4. Jika n lebih dari atau sama dengan 10, fungsi mengambil digit terakhir (n%10)
5. Fungsi kemudian memanggil dirinya sendiri.
6. Proses rekursi terus dilakukan hingga terakhir satu digit.
7. Memanggil fungsi dan menampilkan hasil.

Output :



```
PS C:\pra alpro\minggu 15> python al04.py  
11
```

SOAL 5

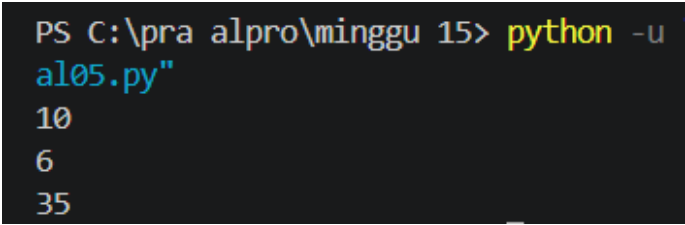
Source Code:

```
def cari_kombinasi(n, k):  
    if k == 0 or k == n:  
        return 1  
  
    return cari_kombinasi(n-1, k-1) + cari_kombinasi(n-1, k)  
  
print(cari_kombinasi(5, 2))  
print(cari_kombinasi(4, 2))  
print(cari_kombinasi(7, 3))
```

Langkah – langkah :

1. Membuat fungsi cari_kombinasi(n, k) untuk menghitung nilai kombinasi.
2. Jika k = 0 atau k = n, fungsi mengembalikan nilai 1.
3. Jika tidak, fungsi menghitung kombinasi dengan rumus rekursif.
4. Proses dilakukan hingga kondisi dasar.
5. Program akan menghitung dan menampilkan hasil kombinasi

Output :



```
PS C:\pra alpro\minggu 15> python -u  
al05.py  
10  
6  
35
```